

T1 — Reutilização de API de Backend (OAuth / Keycloak)

Visão geral

Vamos construir a REST API de OAuth que consome a API REST do Keycloak (autenticação, users, roles). Essa entrega é a fase de "competição" entre os grupos — foco em qualidade, aderência ao enunciado e boas práticas.

Stack: Java + Spring Boot, **WebClient** (reativo) para consumir a Admin API e o endpoint **/token** do Keycloak, Docker/docker-compose.

Decisões arquiteturais e o raciocínio por trás delas

Decisão	Escolha	Por quê
Linguagem/framework	Java + Spring Boot (em vez de Python/FastAPI)	FastAPI seria mais rápido de implementar (menos boilerplate, Swagger automático, setup ágil). Mas o único exemplo de código que o professor forneceu é em Java/Spring, e a disciplina parece esperar esse padrão (testes, SonarQube, arquitetura mais "batida" em Java). O custo de ficar fora do padrão esperado numa disciplina que já mostrou um exemplo de referência pesou mais do que o ganho de velocidade do Python.
Validação de token	Local via JWKS (spring-boot-starter-oauth2-resource-server),	Validação local é o padrão de mercado pra resource server OAuth2/OIDC: valida

Decisão	Escolha	Por quê
	em vez de chamar <code>/userinfo</code> do Keycloak a cada request	assinatura, <code>exp</code> e <code>iss</code> automaticamente, sem round-trip de rede a cada request autenticado. Chamar <code>/userinfo</code> sempre seria mais simples de implementar, mas mais lento e acopla a disponibilidade da nossa API à do Keycloak em todo request. Trade-off aceito: validação local não pega revogação antecipada de token (se um admin desabilitar um token antes de expirar, só vamos perceber quando ele expirar naturalmente) — documentado como decisão consciente.
<code>client_id</code> no login	Fixo via variável de ambiente, não recebido no body do request (mesmo a última versão do enunciado sugerindo isso)	O grant <code>password</code> pressupõe um client confidencial: é a própria API OAuth quem se autentica no Keycloak com <code>client_id/client_secret</code> fixos — o usuário final não deveria escolher isso. Aceitar <code>client_id</code> dinâmico no request abre porta pra alguém tentar se passar por outro client registrado no realm (o Keycloak rejeitaria por falta do secret, mas é um design que convida a esse tipo de erro). Divergência do enunciado documentada e justificada no README — vale confirmar com o professor/monitor.

Decisão	Escolha	Por quê
Contrato de erro	<code>{error_code, error_description, error_source, error_stack}</code> centralizado via <code>@RestControllerAdvice</code>	Exigido pelo enunciado; centralizar evita que cada rota reimplemente o mapeamento de erro do Keycloak pro nosso formato, o que seria fonte garantida de inconsistência/bug.
Rota de refresh	<code>POST /refresh-token</code> dedicada (em vez de reaproveitar <code>/login</code> com um body alternativo)	O enunciado exige tratar a expiração do <code>access_token</code> gerando novos tokens via <code>grant_type: refresh_token</code> . Uma rota separada é mais fácil de documentar no Swagger e deixa a intenção de cada chamada explícita, em vez de um único endpoint com dois comportamentos diferentes dependendo do body.

Divisão do trabalho (4 pessoas)

Arthur — Arquitetura e Infra

Setup do projeto

- ☐ `pom.xml` (dependências: web, webflux, validation, oauth2-resource-server, springdoc-openapi)
- ☐ Estrutura de pacotes (`controller/`, `service/`, `dto/`, `exception/`, `config/`, `validation/`)
- ☐ `Dockerfile` + integração no `docker-compose.yml` do repo base
- ☐ `.env.example` documentado

Configuração central

- ☐ `KeycloakProperties` (base-url, realm, client-id, client-secret)
- ☐ `WebClientConfig`

- ☐ `SecurityConfig` (resource server, validação JWT via JWKS)

Contrato de erro compartilhado (*prioridade — bloqueia as outras pessoas*)

- ☐ `OAuthApiException`
- ☐ `KeycloakErrorMessage`
- ☐ `GlobalExceptionHandler`

Setup de qualidade

- ☐ Config do Swagger (springdoc)
- ☐ Setup do SonarQube
- ☐ Setup do Testcontainers (infra pro teste de integração)

Disponível para: auxiliar qualquer pessoa do grupo no que for necessário (dúvida de Spring, revisão de código, desbloqueio técnico).

João Biasoli — Astah, Postman, Documentação e Apresentação

Astah

- ☐ Diagrama conceitual User/Role da API OAuth
- ☐ (Etapa seguinte) Diagrama conceitual + lógico da API de domínio

Postman

- ☐ Collection com todos os endpoints
- ☐ Environment (`base-api-url`, `base-keycloak-url`, `access_token` automático via script no login)
- ☐ Exportar arquivos pro repositório

Documentação

- ☐ `README.md` (arquitetura adotada, decisões e trade-offs — tabela de raciocínio acima —, instruções de execução)
- ☐ Seção do README linkando pro Swagger

Apresentação

- ☐ Roteiro de demo (login → criar user → listar → atribuir role → refresh → erro tratado)

- ☐ Material de apoio/slides
- ☐ Ensaiar a demo usando a própria collection do Postman

Apoio: Arthur disponível para ajudar a traduzir as decisões técnicas para o README, se precisar.

Pessoa 3 — Autenticação

Endpoints

- ☐ `POST /login` (form-data: username, password, grant_type: password)
- ☐ `POST /refresh-token`

Service

- ☐ `KeycloakAuthService` (chama `/protocol/openid-connect/token` com os dois grant types)

DTOs

- ☐ `LoginRequest`, `RefreshTokenRequest`, `TokenResponse`

Testes

- ☐ Unitários de Auth (mock do Keycloak)

Response codes

- 200 OK / 400 Bad Request / 401 Unauthorized
-

Pessoa 4 — Users e Roles

Endpoints de Users

- ☐ POST, GET (com filtro `?enabled=`), GET/{id}, PUT, PATCH (senha), DELETE (lógico)

Endpoints de Roles

- ☐ CRUD completo de `/roles`

- ☐ POST/DELETE `/users/{id}/roles/{roleId}`

Service

- ☐ `KeycloakUserService` (extrair `id` do header `Location`)
- ☐ `KeycloakRoleService`

DTOs e validação

- ☐ `UserCreateRequest`, `UserUpdateRequest`, `PasswordUpdateRequest`, `UserResponse`
- ☐ `RoleRequest`, `RoleResponse`
- ☐ `EmailValidator` (regex RFC 5322)

Testes

- ☐ Unitários de Users e Roles

Response codes

- Users: 201/400/401/403/409 (criação), 200/400/401/403/404 (demais), 204 (delete)
- Roles: mesma lógica

Revisão cruzada

Não é responsabilidade de uma única pessoa coordenar isso — é regra geral do grupo: **toda parte precisa ser revisada por outra pessoa antes do merge**, sem exceção. Sugestão de como distribuir na prática:

- Se a carga de trabalho de Auth (Pessoa 3) ou de Users/Roles (Pessoa 4) ficar mais tranquila que a dos outros, essa pessoa pode assumir a revisão das demais partes.
 - Caso contrário, fica por conta do grupo se organizar (ex: revisão em pares, ou cada PR pedido pra quem estiver disponível no momento) — o único requisito fixo é que **ninguém faz merge da própria parte sem pedir revisão de alguém**.
-

Testes de integração (compartilhado, no final)

Pessoa 3 e Pessoa 4 fazem juntos, perto do final: sobem o Keycloak via Testcontainers (infra pronta pelo Arthur) e testam o fluxo ponta a ponta (login → criar user → atribuir role → refresh). Roteiro pode reaproveitar o mesmo passo a passo que o João já vai ter montado pra demo.

Ordem de execução

1. **Dia 1:** Arthur sobe o esqueleto (pom.xml, Dockerfile, docker-compose, exception handler básico, WebClientConfig) e faz push — todo mundo puxa essa base.
 2. **Dias seguintes:** Pessoa 3 (Auth), Pessoa 4 (Users/Roles) e João (Asth/Postman/Documentação) trabalham em paralelo, cada um em branch própria.
 3. **Integração:** PRs pro `develop`, revisão cruzada obrigatória (ver seção acima).
 4. **Reta final:** João fecha README/Swagger/apresentação; Pessoa 3 e 4 rodam testes de integração juntos.
-

Pontos em aberto (fora do escopo do T1)

- Domínio da próxima API (classe principal + secundária) e banco (MongoDB vs PostgreSQL) — decidir depois dessa entrega.